

Writing a Cache is Expensive, Reading is Cheaper

The economics of prompt caching

The core principle

In prompt caching, the two operations have **opposite** cost profiles:

- **Writing to the cache** (first time a prefix is sent) is **as expensive as or more expensive than** a normal request
- **Reading from the cache** (every subsequent matching request) is **dramatically cheaper** — typically 50-90% off

This is not a quirk. It is the fundamental design of every major caching system, and it reflects real hardware economics: GPU memory is the scarcest resource in an AI data center, and someone has to pay for it.

Why writes are expensive

When a provider writes a prefix to the cache, they perform **three operations** where a non-cached request would only need **one**:

1. **Run the model normally** on the prompt — compute attention across every token, generate KV tensors. This is the same work a normal request requires.
2. **Save the KV tensors** to fast-tier GPU memory — an extra storage operation.
3. **Hold them there** for the TTL duration — paying real GPU-memory rent for 5 minutes, 1 hour, or longer.

Steps 2 and 3 are pure overhead — work that wouldn't happen at all without caching. The provider passes that overhead on in one of two ways:

- **Charge a write premium** (Anthropic's approach) — the write itself costs more than a normal request
- **Absorb the overhead but offer a smaller read discount** (OpenAI, Google's approach) — writes are billed at standard rates, but reads save less per hit

Either way, the storage cost is reflected somewhere in the pricing. There is no free lunch.

Why reads are cheap

On a cache hit, the provider:

1. **Skips the entire attention computation** for the cached tokens — the most expensive part of inference, by orders of magnitude
2. **Loads the pre-computed KV tensors** directly from GPU memory into the model's attention layers
3. **Only computes attention** for the new (uncached) suffix — usually a tiny fraction of the total prompt

The expensive matrix multiplications for thousands of cached tokens simply do not run. The model conceptually resumes from where it left off on the previous request. That saved compute is what the provider passes back as the read discount.

Concrete numbers across providers

For a **5,000-token prefix** at a base rate of **\$3 per million input tokens**:

Scenario	Cost	Multiplier
No caching (baseline)	\$0.0150	1.00×
Anthropic 5-min write	\$0.0188	1.25×
Anthropic 1-hour write	\$0.0300	2.00×
Anthropic read (any tier)	\$0.0015	0.10×
OpenAI write	\$0.0150	1.00×
OpenAI read	\$0.0075	0.50×
Google Gemini write	\$0.0150	1.00× (+ hourly storage fee)
Google Gemini read	\$0.0038	0.25×

Two patterns emerge from these numbers:

- **Anthropic** charges a write premium but provides the deepest read discount (90% off)
- **OpenAI** and **Google** absorb the write overhead at the moment of writing, but offer smaller read discounts (50% and 75% respectively)

The total economic picture across a workload is similar — the GPU memory has to be paid for somehow, and providers just choose different points at which to bill for it.

The break-even math

Prompt caching is fundamentally an **investment**: pay extra (or the same) up front in exchange for paying much less on every reuse. The question for every prefix is: **will it get reused enough times for the read discount to recoup the write premium?**

The formula is straightforward:

```
Write premium per cache    = premium_pct × prefix_cost
Saved per hit               = discount_pct × prefix_cost
Break-even hits per write  = premium_pct / discount_pct
```

Applied to each provider:

Provider	Premium %	Discount %	Break-even
Anthropic 5-min	25%	90%	0.28 hits per write
Anthropic 1-hour	100%	90%	1.11 hits per write
OpenAI	0%	50%	0 (any hit is profit)
Google Gemini	0%	75%	0 (any hit is profit)

Interpretation: on Anthropic's 5-minute tier, you only need roughly **one cache hit for every four writes** to come out ahead. On the 1-hour tier, you need slightly more than one hit per write — that's why the 1-hour tier only makes sense if you have strong reason to expect bursty reuse with multi-minute gaps. On OpenAI and Google, every hit is pure savings with no down-side risk.

Why this design makes sense

GPU memory near the inference chips is the most expensive resource in an AI data center. A single H100 GPU has 80 GB of high-bandwidth memory (HBM) and costs roughly \$30,000. Holding a 5,000-token prefix in cache for an hour reserves real, expensive hardware. The provider is renting out a slot in their fastest memory tier, and renting fast hardware costs money.

The pricing model reflects honest economics:

- **Writes are the rent payment** — you are buying time in a memory slot
- **Reads are the consumption** — you are using what you have already rented

If the slot gets reused often, the rent amortizes across many cheap reads and you win significantly. If the slot sits unused before expiring, the rent is simply wasted.

The practical takeaway

Prompt caching is not free magic. It is a **contract** between you and the provider: you are telling them "I will send this same prefix again soon, so it is worth holding in fast memory." If you keep that promise, you save 50-90%. If you break that promise, you have paid storage rent for nothing.

That's why traffic patterns matter so much when deciding whether to cache:

Traffic pattern	Typical hit rate	Caching verdict
Continuous chat (turns < 1 min apart)	90%+	Strong win — caching is a no-brainer
Bursty batches (clustered runs)	75%+	Win, possibly with longer TTL
Sparse usage (occasional queries)	~45%	Marginal, depends on prefix size
Cold one-shot queries	~5%	Caching costs you money — skip it

Worked example

Walking through a single prefix end-to-end on Anthropic 5-minute tier, with a 5,000-token prefix at \$3/M input rate.

Scenario A — single isolated request, prefix never reused

Fire 1 (MISS, writes cache):	\$0.0188
Total spent:	\$0.0188
Equivalent uncached cost:	\$0.0150
Net result:	\$0.0038 LOSS

The write premium is wasted because nothing benefits from the cached entry.

Scenario B — request fired twice within 5 minutes

Fire 1 (MISS, writes cache):	\$0.0188
Fire 2 (HIT, reads cache):	\$0.0015
Total spent:	\$0.0203
Equivalent uncached cost:	\$0.0300 (2 × \$0.0150)
Net result:	\$0.0097 SAVED (~32% savings)

After just one reuse, the caching investment has already paid back its premium and started generating real savings.

Scenario C — request fired 10 times within 5 minutes

Fire 1 (MISS, writes cache):	\$0.0188
Fires 2-10 (9 HITS):	9 × \$0.0015 = \$0.0135
Total spent:	\$0.0323
Equivalent uncached cost:	\$0.1500 (10 × \$0.0150)
Net result:	\$0.1177 SAVED (~78% savings)

At scale, caching delivers the dramatic savings it is known for. The write premium becomes a rounding error compared to the cumulative read discount.

Key signals to watch in production

Whether caching is paying off comes down to a few measurable numbers, regardless of provider:

- **Cache hit ratio** — the percentage of input tokens served from cache. Below 30% is a warning sign; above 70% is a healthy production target.
 - **Average prefix size** — bigger prefixes mean bigger savings per hit. Caching a 50,000-token RAG context is much more valuable than caching a 1,200-token system prompt.
 - **Time between requests for the same prefix** — if your traffic pattern has gaps longer than your TTL, you are paying for empty rent.
 - **Write-to-read ratio** — for every cache write, how many reads do you get? Higher is better. A ratio below 1:1 (more writes than reads) means caching is losing you money.
-

Bottom line

Writes are expensive because you are buying memory. Reads are cheap because you are just consuming what you already bought.

Caching pays off when you reuse the memory enough times to amortize the write cost. For most production AI workloads with stable system prompts and consistent traffic, that condition is easy to meet — and the savings can transform a borderline-economical AI feature into a viable product. For one-off queries with no reuse, it is the wrong tool and will cost you money.

The principle to remember: **caching is not a free optimization — it is a deliberate financial bet that prefixes will be reused. The bet pays off handsomely when right, and costs real money when wrong.**